

前项是模糊集合，后项也是模糊集合

区别是后验模糊集（Consequent fuzzy set）的隶属函数是单调函数

这就是一个方法，可以理解为数学算法，分几步完成计算。

Rule 1: IF x_1 is A_1^1 and x_2 is A_2^1 THEN y_1 is B_1
Rule 2: IF x_1 is A_1^2 and x_2 is A_2^2 THEN y_2 is B_2

Advantages:

Tsukamoto model avoids the time consuming process of defuzzification since each rule infers a crisp output and the overall output can be calculated using weighted average method.

Disadvantages:

Since the output membership function should be monotonic in nature, its not as useful as a general approach and hence can only be used in specific situations.

老师发的展示的要求是说明模糊化，规则聚合，推理机制和去模糊策略。（可以理解为计算需要经过的四个步骤）

他们俩制作的word和python代码就分成这四步，计算也是这四步。（我自己觉得这两个文件翻译成中文好理解一点）

目前是将这个方法应用于汽车制动方面。

先用这个汽车制动的例子看这个方法是做什么的。

变量

车速、车距、制动力

车速、车距是自变量，求因变量制动力。（就是不同车速不同距离应该用多大的力刹车）

车速：[0,60]是慢，[60,120]是快，[30,90]是中速。

车距：[0,50]是近，[50,100]是远。

制动力：[0,50]是轻制动，[30,70]是中度制动，[60,100]是强制制动。

计算

（即知道自变量求因变量）

第一步：模糊化

模糊化就是将输入值通过隶属函数转换成隶属度。

首先知道隶属函数 (Membership Function)

隶属函数 ($\mu(x)$) 定义了模糊集合的形状, 告诉我们: 不同的输入值对应多大的“隶属度”。
举个例子。

在同一个车速 (比如 30) 下:

- “快”集合隶属度可能是 0.1
- “中”集合隶属度可能是 0.7
- “慢”集合隶属度可能是 0.3

隶属函数就是帮助找到隶属度。

Tsukamoto System采用的隶属函数是三角隶属函数 (Triangular Membership Function)

三角隶属函数由三个参数 $[a, b, c]$ 定义:

$$\mu(x) = \begin{cases} 0, & x \leq a \\ \frac{x-a}{b-a}, & a < x \leq b \\ \frac{c-x}{c-b}, & b < x < c \\ 0, & x \geq c \end{cases}$$

对于给定变量, 变量数值小, 就用单调递减函数。

$$\mu_{\text{Low}}(x) = \begin{cases} 1, & x \leq a \\ \frac{b-x}{b-a}, & a < x < b \\ 0, & x \geq b \end{cases}$$

变量数值大, 用单调递增函数。

$$\mu_{\text{High}}(x) = \begin{cases} 0, & x \leq a \\ \frac{x-a}{b-a}, & a < x < b \\ 1, & x \geq b \end{cases}$$

在这个汽车制动例子中，对于处于低速范围的用递减函数，处于中速和高速的用递增函数。

若给定速度为75，根据上述函数计算。

先计算u（低速）=0，因为75>30.

u（高速）=（75-60）/（120-60）=0.25

中速求解代码：

```
mu_speed_med = min(
    membership_ascending(30.0, 60.0, speed_x),    # rising to 60
    membership_descending(60.0, 90.0, speed_x)    # falling from 60 to 90
)
```

以60为分界,递增函数区间[30,60],递减函数区间[60,90],求出后取隶属函数最小的。

u（中速）=0.5

对于距离20，同理。

u（近）=0.6

u（远）=0

第二步：规则聚合

规则 IF 条件 THEN 结论

R1 速度高且距离接近制动力强

R2 速度中等且距离接近制动力强

R3 速度高且距离远 制动力中等

R4 速度低且距离接近 制动力中等

R5 速度低且距离远 制动力轻

R6 速度为中速，距离为远制动力轻

自定义，符合常识。

第三步：推理

采用交集计算。模糊集合的交集就是求min

$$\alpha_i = \min (\mu_{A_i}(x), \mu_{B_i}(y))$$

α 是指规则的触发强度。

还是上述速度75，距离20的例子。

R1: $\alpha_1 = \min (\mu_{\text{High}}(\text{速度}), \mu_{\text{Near}}(\text{距离})) = \min (0.25, 0.60) = 0.25$

R2: $\alpha_2 = \min (\mu_{\text{Med}}(\text{速度}), \mu_{\text{Near}}(\text{距离})) = 0.50$ (大约)

求出六个规则的 α ，u（远）和u（低速）=0，所以后四个 α 均为0

第四步：输出映射

根据 α 属于的区间（即制动力），求z

$$z_i = \begin{cases} b - \alpha_i(b - a), & \text{for descending (Light)} \\ a + \alpha_i(b - a), & \text{for ascending (Medium or Strong)} \end{cases}$$

python计算结果：

Rule 1: $\alpha=0.250$, $z=70.00$

Rule 2: $\alpha=0.500$, $z=80.00$

Rule 3: $\alpha=0.000$, $z=30.00$

Rule 4: $\alpha=0.000$, $z=30.00$

Rule 5: $\alpha=0.000$, $z=50.00$

Rule 6: $\alpha=0.000$, $z=50.00$

第五步：去模糊化

去模糊化由加权平均方法来实现。

$$Z^* = \frac{\sum_{i=1}^n (\alpha_i \times z_i)}{\sum_{i=1}^n \alpha_i}$$